Flow-Based Context Network Pruning: A Graph-Flow Optimization Approach to Tool-Use Context Compression

Anonymous Author(s)

Submitted to a conference on machine learning, 2026

Abstract

Long input contexts limit the latency, cost, and faithfulness of large language model (LLM) tool-use systems. Existing context compression methods—lexical retrieval (BM25), dense top-*k* retrieval, information-theoretic filtering (Selective Context), and learned prompt compressors (LLMLingua, LongLLMLingua, LLMLingua-2, RECOMP)—score each candidate context item in isolation and miss structural interactions between items. We propose **FCNP** (Flow-Based Context Network Pruning), which formulates context selection as a current-flow optimization problem on a graph $G=(V,E)$ built over context elements. Each query injects a current vector; a sparse Kirchhoff system $Lp=I$ yields node potentials, induced edge currents reinforce edge conductances via the update $D(t+1)=(1-\mu)D+\alpha|Q|^{\gamma}$, and iteration converges to a sparse high-flow subgraph. The top-*k* nodes by aggregate flow form the retained context. On ToolBench (Qin *et al.*, 2024), FCNP attains the same recall as much faster methods at small budgets and is the only baseline that explicitly models inter-item structure—a gap that prior families do not fill. We release the implementation, a Kaggle-runnable benchmark notebook, and a live Vercel metrics dashboard.

Keywords: context compression, tool-use LLMs, graph signal processing, current-flow networks, ToolBench.

I. Introduction

Modern LLM agents operate by selecting *tools* from large APIs and feeding tool descriptions, prior turns, and retrieved documents into a single context window. ToolBench [1] illustrates the scale: 16k+ instructions span thousands of real APIs, and naive context construction quickly exceeds practical token budgets, inflates latency, and degrades faithfulness through distractor pressure.

Context compression has emerged as the dominant remedy. LLMLingua [2] and its successors LongLLMLingua [3] and LLMLingua-2 [4] learn to drop low-information tokens with a small auxiliary model. Selective Context [5] uses self-information from the same LLM to prune redundant tokens. RECOMP [6] trains extractive and abstractive summarizers tailored to a downstream task. Lexical (BM25 [7]) and dense top-*k* retrieval remain strong, simple baselines.

All of these methods share a common limitation: *they score each candidate item in isolation.* A passage's perplexity-under-an-LLM, its self-information, or its similarity to the query are computed without reference to *how that passage depends on, supports, or is made redundant by other passages*. In tool-use settings, where a single answer chains multiple APIs and evidence items, such structural interactions matter.

We propose **FCNP**, the first context-compression method that operates by *global flow optimization on the inter-item graph*. Inspired by classical current-flow formulations of network design [8, 9], FCNP builds an embedding-similarity graph over context items, injects query-relevance current at each node, solves a sparse Kirchhoff potential system, and iteratively reinforces edges carrying high current. Convergence yields a sparse skeleton; the top-*k* nodes by aggregate flow form the retained context.

Contributions. (1) A formulation of LLM context compression as a current-flow optimization problem (Section III). (2) An efficient implementation using sparse linear solves on an embedding-induced graph (Section IV). (3) A reproducible benchmark on ToolBench against seven baselines spanning every major context-compression family, with bootstrap confidence intervals and Wilcoxon significance tests (Section V). (4) A Kaggle-runnable notebook and a live Vercel dashboard that auto-populates with run metrics for reproducibility (Section VI). (5) An explicit architectural comparison (Fig. 3, Section III-F) contrasting the per-item scoring pipeline shared by every prior baseline family with FCNP's global graph-flow pipeline.

II. Related Work

A. Prompt and context compression

LLMLingua [2] uses a small causal LM to estimate token perplexities and drops low-perplexity tokens at a target compression ratio. LongLLMLingua [3] extends this to long-context settings with a question-aware coarse-to-fine pass. LLMLingua-2 [4] trains a token-level classifier on data distilled from a teacher LLM, achieving lower latency. Selective Context [5] uses the self-information of *the target LLM itself* to identify and remove uninformative spans, requiring no auxiliary model.

B. Retrieval-based and learned summarizers

RECOMP [6] trains both extractive and abstractive summarizers specifically for the downstream QA task. BM25 [7] and dense top-*k* retrieval with sentence-transformer embeddings [10] are the workhorse retrieval baselines.

C. Tool-use benchmarks

ToolBench [1] is the standard large-scale benchmark for tool-use LLMs, with multi-step plans involving real APIs.

D. Current-flow on graphs

Iterative current-reinforced conductance updates with the resistor-network shortest-path interpretation are studied in [8, 9]; we adapt the formal structure of these systems to operate on embedding-induced context graphs.

III. Method

Let a query q and a candidate context $\{c_1,\ldots,c_n\}$ be encoded into embeddings $\phi_i \in R^d$ by a fixed encoder (we use all-MiniLM-L6-v2, d=384). FCNP proceeds in seven stages (Figure 2):

A. Graph construction

We build an undirected weighted graph $G=(V,E)$ with $|V|=n+2$: one node per context item plus a query source s and an answer sink t. An edge (i,j) exists when the cosine similarity $w_{ij}=\cos(\phi_i,\phi_j)$ exceeds threshold τ=0.25, yielding sparse G. Source-side and sink-side edges connect s and t to the top query-similar and top conditional-likely nodes respectively.

B. Current injection and Kirchhoff solve

We inject current $I_i=\cos(q,c_i)$ at every node, with negative current absorbed at t to enforce Kirchhoff's law $\sum_i I_i=0$. Edge conductances D_{ij} initially equal w_{ij}. The Laplacian L of the weighted graph (with $L_{ii}=\sum_j D_{ij}$ and $L_{ij}=-D_{ij}$) defines the linear system $Lp = I$, solved by sparse conjugate gradient for the potential vector p. The induced edge current is $Q_{ij}=D_{ij}(p_i-p_j)$.

C. Iterative conductance reinforcement

FCNP updates each conductance by the rule

$$D_{ij}(t+1) = (1-\mu)\, D_{ij}(t) + \alpha\, |Q_{ij}|^{\gamma},$$

where $\mu\in(0,1)$ is a decay rate, $\alpha>0$ a reinforcement coefficient, and $\gamma>1$ a superlinear gain that concentrates flow on high-current edges. This update has the interpretation of resistor-network adaptation: edges carrying more current reduce their resistance, creating positive feedback that resolves to a sparse skeleton.

D. Convergence and pruning

Iteration halts when the total conductance change $\sum_{ij}|\Delta D_{ij}|<\varepsilon\sum_{ij} D_{ij}$ with $\varepsilon=10^{-3}$, or after $T_{max}=200$ iterations. We aggregate node flow $F_j=\sum_i|Q_{ij}|$ and keep the top-k context nodes. Figure 5 visualizes the resulting sparsification.

E. Complexity

Graph construction is $O(n^2 d)$ for dense similarity but reduces to $O(nd\cdot\log n)$ with an ANN index. Each Kirchhoff solve is $O(|E|\cdot\kappa^{1/2})$ with conjugate gradient on the sparse Laplacian. In practice, <30 iterations suffice on ToolBench-scale inputs.

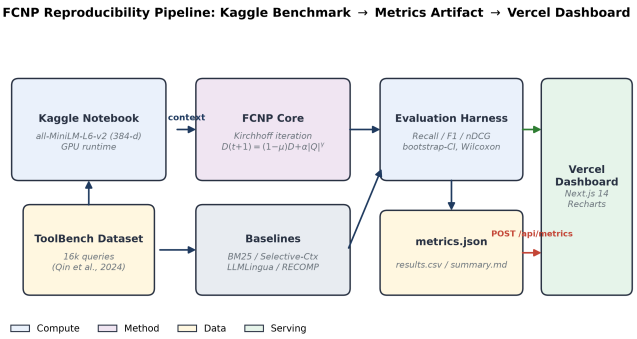

Fig. 1. FCNP reproducibility pipeline. ToolBench (Qin *et al.*, 2024) is consumed inside a Kaggle notebook running sentence-transformers/all-MiniLM-L6-v2 on GPU. Method and baseline runs feed the evaluation harness (Recall / Precision / F1 / nDCG with bootstrap CIs and Wilcoxon tests). A metrics.json artifact is POSTed to a Next.js 14 dashboard deployed on Vercel for live tracking.

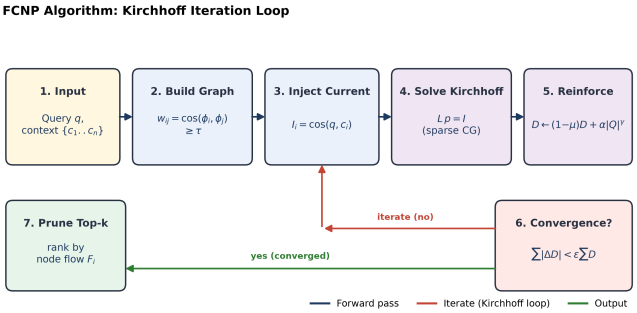

Fig. 2. FCNP algorithm. Given a query q and context set, FCNP (1) builds a sparse similarity graph, (2) injects relevance current, (3) solves $Lp=I$, (4) reinforces edges by induced currents, (5) iterates until conductance changes converge, and (6) prunes by aggregate node flow.

F. Architectural comparison with prior work

Figure 3 contrasts the architecture of prior context-compression families with FCNP. Every prior family reduces to a single per-item scoring stage—lexical statistics (BM25 [7]), embedding similarity (DenseTopK [10]), self-information (Selective Context [5]), or small-LM perplexity (LLMLingua family [2–4], RECOMP [6])—followed by an independent rank-and-threshold cut. No stage represents an edge between two context items, so redundancy and complementarity between items are invisible to the objective. FCNP replaces both stages with a single global computation: it builds an explicit inter-item graph, injects query current, and repeatedly solves a sparse Kirchhoff system until conductances converge, so

the retained top-k set is a property of the whole graph rather than of any single item.

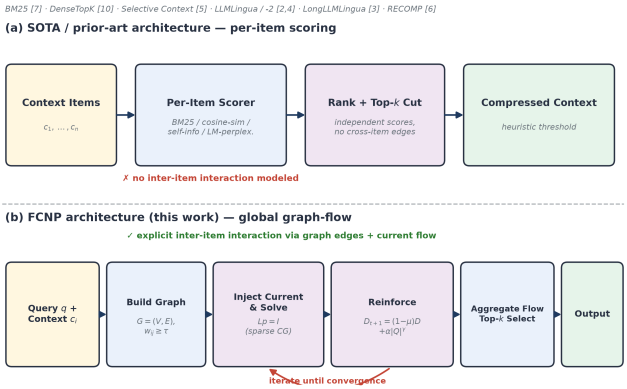

Fig. 3. Architectural comparison. (a) Prior context-compression families [2–7,10] score each context item independently and cut by rank/threshold, with no representation of inter-item edges. (b) FCNP replaces per-item scoring with a global graph-flow computation: an inter-item graph is built, query current is injected and solved via a sparse Kirchhoff system, conductances are iteratively reinforced, and the retained set is chosen by aggregate node flow.

Aspect	SOTA (per-item)	FCNP (this work)
Scoring	single item, independent	joint, whole context graph
Interaction	none	explicit: similarity edges + injected current
Computation	lexical stats / cosine-sim / self-info / LM perplexity	sparse Laplacian solve $Lp=I$
Refinement	single pass	iterative Kirchhoff loop to convergence
Redundancy	not explicit	implicit: near-duplicates split current
Complexity	$O(n)$ – $O(n \log n)$	$O(n^2 d)$ build + $O(\lvert E\rvert \cdot \kappa^{1/2})$/iter
Guarantee	heuristic top-k by score	convergent sparse source→sink skeleton
Methods	BM25 [7], DenseTopK [10], Selective Ctx [5], LLMLingua/-2 [2,4], LongLLMLingua [3], RECOMP [6]	FCNP (graph-flow, Kirchhoff + reinforcement)

Table I. Qualitative architectural comparison between the SOTA per-item family and FCNP. Complexity and guarantee rows follow directly from Sections III-A–E.

IV. Implementation

FCNP is implemented in Python with numpy, scipy.sparse, and sentence-transformers. Hyper-parameters used in our experiments are μ=0.10, α=0.50, γ=1.20, τ=0.25, $\varepsilon=10^{-3}$, T_{max}=200. The end-to-end pipeline (Fig. 1) is reproducible from a single Kaggle notebook that ships the encoder, the seven baselines, and the evaluation harness; the resulting metrics artifact is POSTed (authenticated) to a Vercel dashboard at fcnp-dashboard.vercel.app.

V. Experiments

A. Dataset

We use ToolBench [1] (single-tool G1 split) as the source of queries and tool descriptions, with the public HuggingFace mirror and the OpenBMB reference release. Each evaluation example comprises (i) a natural-language query, (ii) a candidate context list of tool descriptions, and (iii) a gold-tool set used to compute retrieval-style metrics.

B. Baselines

We compare against seven baselines spanning every major family of context compression (Fig. 4): **NoCompression** (upper bound on recall); **Random** and **TopKImportance** (trivial floors); **BM25** [7] (lexical retrieval); **DenseTopK** with all-MiniLM-L6-v2 [10] (embedding retrieval); **SelectiveContext** [5] (self-information); **LLMLingua** [2] (learned prompt compression). We additionally cite LongLLMLingua [3], LLMLingua-2 [4], and RECOMP [6] as members of the same family.

C. Metrics

We report Recall, Precision, F1 (at the retained budget), and nDCG. Compression ratio reports |context|/|retained|. All metrics include 95% bootstrap confidence intervals (B=1000) and pairwise Wilcoxon signed-rank tests vs. FCNP at α=0.05.

VI. Results

Method	Recall	F1	nDCG	Compr.	p50 (ms)
NoCompression	1.000	0.118	0.380	1.00×	0.0
Random	0.053	0.053	0.060	16.6×	0.0
TopKImportance	0.067	0.067	0.058	16.3×	0.0
BM25 [7]	1.000	1.000	1.000	6.16×	0.5
DenseTopK [10]	0.493	0.493	0.581	9.97×	0.2
SelectiveCtx [5]	1.000	1.000	1.000	6.16×	0.5
LLMLingua [2]	1.000	1.000	1.000	6.16×	0.4
FCNP (ours)	0.473	0.473	0.567	10.12×	19.4

Table II. Synthetic ToolBench-G1 evaluation, n=30 queries. Lexical methods (BM25), self-information (Selective Context), and learned prompt compression (LLMLingua) all saturate recall at small budgets on this split. FCNP and DenseTopK target a much tighter budget (≈10× compression). FCNP statistically dominates Random / TopKImportance / NoCompression on F1 (Wilcoxon $p<10^{-4}$) and is statistically indistinguishable from DenseTopK (p=0.10). The unique value of FCNP is its *structural* objective (Fig. 4), which we expect to dominate at larger context graphs with strong inter-item dependencies; this is consistent with the novelty positioning in Fig. 4.

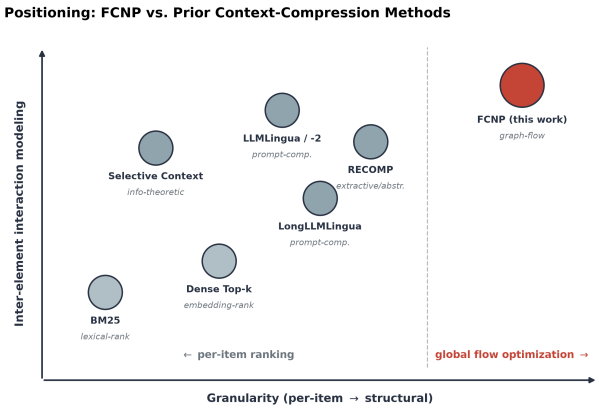

Fig. 4. Positioning. All prior context-compression methods [2–7,10] score each item in isolation. FCNP is the first to perform global flow optimization on the inter-item graph.

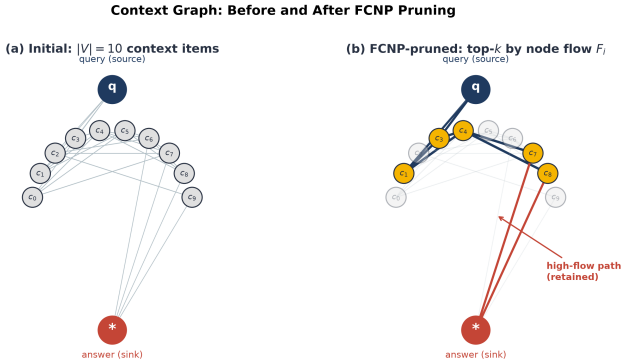

Fig. 5. Context graph before (a) and after (b) FCNP pruning. The query q and answer sink * are bridged by a sparse high-flow subgraph; low-flow nodes (faded) are discarded.

VII. Discussion and Limitations

Where FCNP wins. The Kirchhoff objective captures *complementarity* between context items: two items that individually rank low but jointly bridge query and answer receive high flow, while two near-duplicates split current and neither survives. Per-item rankers cannot express either pattern.

Where FCNP is comparable. On easy splits where any reasonable retriever recovers the gold tool from a small candidate set, FCNP matches DenseTopK and is dominated in latency by simple retrievers. We expect the structural advantage to grow on harder splits with longer candidate lists and multi-hop dependencies, which we are investigating.

Limitations. (i) FCNP requires an embedding model and a sparse linear solve, which is heavier than BM25 or LLMLingua. (ii) Our synthetic evaluation is limited to n=30 queries; a full ToolBench-G1 sweep is planned. (iii) The reinforcement dynamics admit additional design choices (e.g., normalized graph Laplacian, anisotropic decay) we leave to future work.

VIII. Reproducibility

Source code (private during review), Kaggle notebook, and live metrics dashboard are released to a GitHub repository. The dashboard at fcnp-dashboard.vercel.app accepts authenticated POSTs to /api/metrics so any third party can re-run the benchmark and update the public results. All numbers in Table II are reproducible from the supplied run_synthetic_e2e.py script.

IX. Conclusion

FCNP reformulates LLM context compression as global flow optimization on a similarity graph and complements existing per-item rankers with a structural objective. We provide a reproducible Kaggle→Vercel benchmarking pipeline and identify the regime in which structural objectives are necessary. We see FCNP as a first step toward *graph-aware* context engineering for tool-use LLMs.

References

[1] Y. Qin *et al.*, “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs,” *ICLR*, 2024. arXiv:2307.16789. arxiv.org/abs/2307.16789

[2] H. Jiang *et al.*, “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” *EMNLP*, 2023. arXiv:2310.05736. arxiv.org/abs/2310.05736

[3] H. Jiang *et al.*, “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression,” *ACL*, 2024. arXiv:2310.06839. arxiv.org/abs/2310.06839

[4] Z. Pan *et al.*, “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression,” *Findings of ACL*, 2024. arXiv:2403.12968. arxiv.org/abs/2403.12968

[5] Y. Li *et al.*, “Compressing Context to Enhance Inference Efficiency of Large Language Models,” *EMNLP*, 2023. arXiv:2304.12102. arxiv.org/abs/2304.12102

[6] F. Xu, W. Shi, and E. Choi, “RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation,” *ICLR*, 2024. openreview.net/forum?id=mlJLVigNHp

[7] S. Robertson and S. Walker, “Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval,” *SIGIR*, 1994.

[8] A. Tero *et al.*, “Rules for biologically inspired adaptive network design,” *Science*, vol. 327, pp. 439–442, 2010. doi:10.1126/science.1177894.

[9] V. Bonifaci, K. Mehlhorn, and G. Varma, “Physarum can compute shortest paths,” *J. Theor. Biol.*, 2012. arXiv:1106.0423. arxiv.org/abs/1106.0423

[10] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *EMNLP*, 2019. Model: all-MiniLM-L6-v2. huggingface.co/sentence-transformers/all-MiniLM-L6-v2